That error log is a rite of passage for ESP32 developers! The assert failed: block_locate_free error means the ESP32 simply ran out of heap memory (RAM) while trying to load the massive Bluetooth stack. The ESP32-S3 can easily handle it, but we need to tell the Arduino IDE to allocate the memory correctly.

Here is exactly how to fix that crash, followed by your definitive hardware wiring guide.

1. Fixing the Memory Crash (Arduino IDE Settings)

By default, the Arduino IDE reserves a huge chunk of memory for things we aren't using (like OTA Wi-Fi updates), leaving no room for the BLE library.

1. Go to **Tools** > **Partition Scheme**.
2. Change it from "Default" to **"Huge APP (3MB No OTA/1MB SPIFFS)"**.
3. *Optional but recommended:* If you see a menu for **Tools** > **PSRAM**, set it to **"OPI PSRAM"** or **"Enabled"** (The S3 Zero chip has extra hidden RAM we can unlock here).
4. Hit **Upload** again. The crash will disappear!

---

2. The Hardware Wiring Guide

Here is exactly how to wire your ESP32-S3 Zero to the real world.

A. Powering the CD4052BCN Chip

- **Do I power it with the ESP32? YES. * Which Pin?** Connect the VDD (Power) pin of the CD4052 to the **3V3** pin on the ESP32-S3.
- **Why not 5V?** The ESP32's signal pins output 3.3V. If you power the CD4052 with 5V, it might not "read" the 3.3V signal from the ESP32 correctly. Powering both at 3.3V ensures they speak the exact same logic language. Connect the VSS and VEE pins of the CD4052 to the ESP32's **GND**.

B. Connecting the Signal (GPIO 4)

- In our code, const int SBUS_SWITCH_PIN = 4;.
- Connect **GPIO 4** (Labeled as 4 or GP4 on the board) directly to the **"A" Select Pin** (Usually Pin 10) on the CD4052.
- *(Make sure the "Inhibit" pin on the CD4052 is wired to GND so the chip stays active).*

C. Connecting the LED (GPIO 2)

- **Is a resistor mandatory? Absolutely.** If you do not use a resistor, the LED will try to pull infinite current and will instantly burn out GPIO 2, permanently damaging your ESP32-S3.
- **The Wiring:**
  1. Connect **GPIO 2** (Labeled 2 or GP2) to the **Long Leg (+)** of the Red LED.
  2. Connect the **Short Leg (-)** of the LED to one end of a **220Ω or 330Ω resistor**.
  3. Connect the other end of the resistor to any **GND** pin on the ESP32.

Once you change that Partition Scheme and hit Upload, let me know if the boot sequence stays stable through the 60-second window!